

SE3040 – Application Frameworks

Testing Report

Evaluation 02

ReWear – Apparel Swap Platform

Project	ReWear – Apparel Swap Platform
Repository	https://github.com/nuwanakanadil/demo.git
Prepared By	SE-43

1. Overview

Purpose: This report organizes the current project test cases into a clean, submission-ready format for Evaluation 02. It groups unit, integration, and performance testing by module for easier review during marking and viva.

Scope: The report includes the latest documented test cases for Swap, Apparel, Review, Chat, Auth, Wishlist, Admin, and overall system performance. Earlier duplicate or contradictory legacy test entries were cleaned where appropriate to keep the report consistent.

2. Testing Environment and Execution

Testing was carried out on the ReWear full-stack application using a Node.js and React development environment. Functional verification covered backend API behavior, authentication, CRUD operations, user interactions, and administrative workflows.

Unit tests were used to validate individual functions, controllers, or middleware behaviors in isolation. Integration tests were used to verify end-to-end route behavior, including request handling, authorization, and response validation. Performance tests evaluated API stability under concurrent load using request bursts and repeated access patterns.

Where relevant, Artillery-style load testing scenarios were used to evaluate backend responsiveness under simultaneous requests.

3. Test Coverage Summary

Module	Unit Tests	Integration Tests	Performance Tests	Total
Swap	10	10	0	20
Apparel	10	16	0	26
Review	7	9	0	16
Chat	3	8	0	11
Auth	10	0	0	10
Wishlist	5	4	0	9
Admin	5	26	2	33
System	0	0	5	5

4. Swap Module

The following tables summarize the documented test cases for the swap module based on the latest project test case set.

Unit Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
1	createSwap()	requesterId: <user_1>, requestedItemId: <item_1>, offeredItemId: <item_2>, message: Interested in swap	Swap created successfully with status PENDING	swap created, requester: <user_1>, owner: <owner_id>, status: PENDING	Pass
2	createSwap()	requestedItemId missing,	Exception thrown:	Exception thrown:	Pass

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
		offeredItemId: <item_2>	requestedItemId and offeredItemId are required.	requestedItemId and offeredItemId are required.	
3	createSwap()	requester tries to request own item	Exception thrown: You cannot request your own item.	Exception thrown: You cannot request your own item.	Pass
4	createSwap()	offered item does not belong to requester	Exception thrown: You can only offer an item that you own.	Exception thrown: You can only offer an item that you own.	Pass
5	createSwap()	duplicate pending swap for same requester and requested item	Exception thrown: You already have a pending swap request for this item.	Exception thrown: You already have a pending swap request for this item.	Pass
6	getByIdForViewer()	swapId: <swap_id>, viewerId: <owner_id>, viewerRole: user	Swap details returned successfully	swap object returned with requester, owner, requestedItem, offeredItem, logistics	Pass
7	updateLogistics()	swapId: <accepted_swap>, userId: <participant_id>, method: MEETUP, meetupLocation: Colombo, meetupAt: valid date	Logistics updated successfully, status becomes IN_TRANSIT	logistics.method: MEETUP, logistics.meetupLocation: Colombo, logistics.status: IN_TRANSIT	Pass
8	acceptSwap()	swapId: <pending_swap>, ownerId: <owner_id>	Swap status updated to ACCEPTED, requested and offered items marked unavailable	status: ACCEPTED, requestedItem.isAvailable: false, offeredItem.isAvailable: false	Pass
9	rejectSwap()	swapId: <pending_swap>, ownerId: <owner_id>	Swap status updated to REJECTED	status: REJECTED	Pass
10	completeSwap()	swapId: <accepted_swap>, actorId: <requester_id>	Swap status updated to COMPLETED, ownership of items exchanged, logistics status becomes DONE	status: COMPLETED, requestedItem.owner changed, offeredItem.owner changed, logistics.status: DONE	Pass

Integration Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
1	/api/swaps POST	requestedItemId: <item_1>, offeredItemId: <item_2>, message: Interested in swap	201 Created with swap data, status PENDING	201 Created, success: true, data: { requestedItem: <item_1>, offeredItem: <item_2>, status: PENDING }	Pass
2	/api/swaps POST	requester tries to request own item	400 Error, You cannot request your own item.	400 Error, message: You cannot request your own item.	Pass
3	/api/swaps/incoming GET	authenticated owner user	200 OK with incoming swap requests	200 OK, success: true, data: [swap_1, swap_2]	Pass
4	/api/swaps/outgoing GET	authenticated requester user	200 OK with outgoing swap requests	200 OK, success: true, data: [swap_1]	Pass

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
5	/api/swaps/:id/logistics GET	swapId: <swap_id>, requester or owner as viewer	200 OK with logistics details	200 OK, success: true, data: { logistics: { method: MEETUP } }	Pass
6	/api/swaps/:id/logistics PUT	swapId: <accepted_swap>, method: DELIVERY, deliveryOption: Courier, deliveryAddress: 123 Main St, phoneNumber: 0771234567, trackingRef: TRK001	200 OK with updated logistics, status IN_TRANSIT	200 OK, success: true, data: { logistics: { method: DELIVERY, trackingRef: TRK001, status: IN_TRANSIT } }	Pass
7	/api/swaps/:id/accept PUT	swapId: <pending_swap>, authenticated owner	200 OK, swap status updated to ACCEPTED	200 OK, success: true, data: { status: ACCEPTED }	Pass
8	/api/swaps/:id/reject PUT	swapId: <pending_swap>, authenticated owner	200 OK, swap status updated to REJECTED	200 OK, success: true, data: { status: REJECTED }	Pass
9	/api/swaps/:id/complete PUT	swapId: <accepted_swap>, authenticated requester	200 OK, swap status updated to COMPLETED	200 OK, success: true, data: { status: COMPLETED }	Pass
10	/api/swaps/:id/accept PUT	no auth token	401 Unauthorized	401 Error, message: Unauthorized	Pass

5. Apparel Module

The following tables summarize the documented test cases for the apparel module based on the latest project test case set.

Unit Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
1	/apparel POST	title: Shirt, size: M, isAvailable: true	201 Created with item data	201 Created, success: true, data: {title: Shirt}	Pass
2	/apparel POST	title: Shirt	500 Error due to missing size	500 Error, message: title and size are required.	Pass
3	/apparel GET	none	200 OK with paginated list	200 OK, data: [{title: Shirt}], pagination: {page: 1, limit: 10}	Pass
4	/apparel/:id GET	id: a1	200 OK with item data	200 OK, data: {title: Shirt}	Pass
5	/apparel/:id GET	id: a2	404 Error: Item not found	404 Error, message: Item not found	Pass
6	/apparel PUT	id: a1, title: Updated Shirt, images: [{url: "http://img.com/1.jpg"}]	200 OK with updated item data	200 OK, data: {title: Updated Shirt, images: [img.com/1.jpg]}	Pass
7	/apparel PUT	id: a1, title: Updated Shirt	403 Error: Not owner	403 Error, message: You can only update your own item.	Pass
8	/apparel DELETE	id: a1	200 OK, item deleted	200 OK, message: Item deleted	Pass
9	/apparel DELETE	id: a1	403 Error: Not owner	403 Error, message: You can only delete your own item.	Pass
10	/my-items GET	none	200 OK, return items of logged-in user	200 OK, data: [{title: Shirt}]	Pass

Integration Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
1	/api/items POST	title: Black Jacket, description: Stylish jacket, category: OUTERWEAR, size: L, condition: LIKE_NEW	201 Created with the item data	201 Created, title: Black Jacket, category: OUTERWEAR, size: L, condition: LIKE_NEW	Pass
2	/api/items POST	description: Missing title, size: M	400 Error, title and size are required	400 Error, message: title and size are required	Pass
3	/api/items POST	title: Missing Size Item, description: No size	400 Error, title and size are required	400 Error, message: title and size are required	Pass
4	/api/items GET	none	200 OK with list of items	200 OK, success: true, items: [Blue Hoodie, Sneakers], pagination: {total: 2}	Pass
5	/api/items GET	category: TOP	200 OK with filtered items by category	200 OK, success: true, items: [Blue Hoodie]	Pass
6	/api/items GET	available: true	200 OK with filtered items by availability	200 OK, success: true, items: [Blue Hoodie], all items available	Pass
7	/api/items/:id GET	id: <item_id>	200 OK with item data	200 OK, success: true, data: {title: Blue Hoodie, size: M, condition: GOOD}	Pass
8	/api/items/:id GET	id: fakeld	404 Error, Item not found	404 Error, message: Item not found	Pass
9	/api/items/:id PUT	title: Updated Blue Hoodie, description: Updated description, category: TOP, size: L, condition: LIKE_NEW, isAvailable: false	200 OK with updated item data	200 OK, title: Updated Blue Hoodie, isAvailable: false	Pass
10	/api/items/:id PUT	title: Hacked Item	403 Error, You can only update your own item	403 Error, message: You can only update your own item	Pass
11	/api/items/:id PUT	id: fakeld, title: Updated Missing Item	404 Error, Item not found	404 Error, message: Item not found	Pass
12	/api/items/:id DELETE	id: <item_id>	200 OK, Item deleted	200 OK, success: true, message: Item deleted	Pass
13	/api/items/:id DELETE	id: <item_id>	403 Error, You can only delete your own item	403 Error, message: You can only delete your own item	Pass
14	/api/items/:id DELETE	id: fakeld	404 Error, Item not found	404 Error, message: Item not found	Pass
15	/api/items/me/my GET	user_id: <owner_id>	200 OK, Return only my items	200 OK, success: true, items: [Blue Hoodie, Owner Shirt]	Pass
16	/api/items POST	unauthorized request	401 Error, Unauthorized	401 Error, message: Unauthorized	Pass

6. Review Module

The following tables summarize the documented test cases for the review module based on the latest project test case set.

Unit Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
1	/reviews/:userId GET	userId: owner1	200 OK with average rating and review count	200 OK, avgRating: 4.5, count: 2	Pass
2	/reviews/:userId GET	userId: owner1	200 OK with 0 avg rating if no reviews	200 OK, avgRating: 0, count: 0	Pass
3	/reviews/:userId POST	rating: 5, comment: Nice	400 Error: itemId is required	400 Error, message: itemId is required	Pass
4	/reviews/:userId POST	rating: 6, comment: Nice, itemId: i1	400 Error: Rating must be 1-5	400 Error, message: Rating must be 1-5	Pass
5	/reviews/:userId POST	rating: 4, comment: blank, itemId: i1	400 Error: Comment is required	400 Error, message: Comment is required	Pass
6	/reviews/:userId POST	rating: 4, comment: Nice, itemId: i1	400 Error: You cannot review yourself	400 Error, message: You cannot review yourself	Pass
7	/reviews/:userId POST	rating: 5, comment: Nice, itemId: i1	409 Conflict: Already reviewed	409 Conflict, message: You already reviewed this owner for this item	Pass

Integration Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
1	/api/users/:userId/reviews POST	itemId: <item_id>, rating: 5, comment: Very trustworthy owner	201 Created with review data	201 Created, success: true, rating: 5, comment: Very trustworthy owner	Pass
2	/api/users/:userId/reviews POST	itemId: <item_id>, rating: 4, comment: Updated review	201 Created, review updated	201 Created, success: true, rating: 4, comment: Updated review	Pass
3	/api/users/:userId/reviews GET	userId: <owner_id>	200 OK with review summary	200 OK, success: true, avgRating: 4.5, count: 2, reviews: [{rating: 4, comment: Good owner}, {rating: 5, comment: Excellent owner}]	Pass
4	/api/users/:userId/reviews GET	userId: <owner_id>	200 OK with empty review summary	200 OK, success: true, avgRating: 0, count: 0, reviews: []	Pass
5	/api/users/:userId/reviews POST	itemId: <item_id>, rating: 5, comment: Self review	400 Error, You cannot review yourself	400 Error, success: false, message: You cannot review yourself	Pass
6	/api/users/:userId/reviews POST	itemId: <item_id>, rating: 10, comment: Invalid rating	400 Error, Rating must be 1-5	400 Error, success: false, message: Rating must be 1-5	Pass
7	/api/users/:userId/reviews POST	itemId: <item_id>, rating: 4, comment: blank	400 Error, Comment is required	400 Error, success: false, message: Comment is required	Pass

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
8	/api/users/:userId/reviews POST	rating: 4, comment: Missing item id	400 Error, itemId is required	400 Error, success: false, message: itemId is required	Pass
9	/api/users/:userId/reviews POST	itemId: <item_id>, rating: 5, comment: No auth	401 Unauthorized	401 Error, success: false, message: Unauthorized	Pass

7. Chat Module

The following tables summarize the documented test cases for the chat module based on the latest project test case set.

Unit Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
1	/chat/send/:conversationId POST	conversationId: conv1, text: Hello, clientMessageId: 123	201 Created with success message	201 Created, success: true, message sent	Pass
2	/chat/messages/:conversationId GET	conversationId: conv1	200 OK with list of messages	200 OK, success: true, messages: [{text: Hi}, {text: Hello}]	Pass
3	/chat/conversation POST	itemId: item1, ownerId: user999	200 OK with success message	200 OK, success: true, conversation created or fetched	Pass

Integration Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
1	/api/chats/conversations POST	itemId: <item_id>, ownerId: <owner_id>	200 OK with conversation data	200 OK, success: true, itemId: <item_id>, participants: [<buyer_id>, <owner_id>]	Pass
2	/api/chats/conversations POST	itemId: <item_id>, ownerId: <owner_id>	200 OK, no duplicate conversations created	200 OK, success: true, only 1 conversation created	Pass
3	/api/chats/conversations/:id/messages GET	conversationId: <conversation_id>	200 OK with messages in ascending order	200 OK, success: true, messages: [Hi, Hello]	Pass
4	/api/chats/conversations/:id/messages POST	text: Is this still available?, clientMessageId: client-msg-1	201 Created with new message and updated conversation preview	201 Created, success: true, message: Is this still available?, conversation updated	Pass
5	/api/chats/conversations GET	none	200 OK with list of conversations, item preview, and unread count	200 OK, success: true, conversations: [Winter Coat], unreadCount: 1	Pass
6	/api/chats/conversations/:id/read POST	conversationId: <conversation_id>	200 OK, conversation marked as read	200 OK, success: true, lastReadAt: <timestamp>	Pass
7	/api/chats/conversations/:id/read	conversationId: fakeId	404 Error, Conversation not found	404 Error, message: Conversation not found	Pass

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
	POST				
8	/api/chats/conversations GET	none	401 Unauthorized due to missing auth header	401 Error, message: Unauthorized	Pass

8. Auth Module

The following tables summarize the documented test cases for the auth module based on the latest project test case set.

Unit Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
1	/register POST	name: John, email: john@test.com, password: password123	201 Created with success message and verification email	201 Created, success: true, user created with verification email	Pass
2	/register POST	name: John, email: john@test.com, password: password123	400 Error: Email already exists	400 Error, message: Email already exists	Pass
3	/login POST	email: test@test.com, password: password123	200 OK with token	200 OK, success: true, token: <generated_token>	Pass
4	/forgot-password POST	email: fake@test.com	200 OK, success message	200 OK, success: true, success message returned	Pass
5	/reset-password POST	token: token123, newPassword: newpass123, confirmPassword: newpass123	200 OK, success message	200 OK, success: true, password reset successfully	Pass
6	/reset-password POST	token: token123, newPassword: a, confirmPassword: b	400 Error, passwords do not match	400 Error, message: Passwords do not match	Pass
7	/me GET	none	200 OK, return current user data	200 OK, success: true, user data returned	Pass
8	/verify-email GET	token: token123	200 OK, email verified	200 OK, success: true, email verified	Pass
9	/delete-me DELETE	none	200 OK, success message	200 OK, success: true, user deleted	Pass
10	/update-me PATCH	name: New Name, newPassword: newpass, confirmPassword: newpass	200 OK, user updated	200 OK, success: true, user updated	Pass

9. Wishlist Module

The following tables summarize the documented test cases for the wishlist module based on the latest project test case set.

Unit Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
1	addToWishlist()	itemId:<item_id>, userId:<user_id>	Item successfully added to wishlist	Item added, wishlist size increased	Pass
2	addToWishlist()	itemId:<item_id> (duplicate), userId:<user_id>	Duplicate item not added	Duplicate prevented, no change in wishlist	Pass
3	getWishlist()	userId:<user_id>	List of wishlist items returned	[T-shirt, Jacket]	Pass
4	removeFromWishlist()	wishlistItemId:<wishlist_id>	Item removed successfully	Item removed, wishlist updated	Pass
5	matchWishlistItems()	userId:<user_id>	Matching items available for swap	[Denim Jacket]	Pass

Integration Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
1	/api/wishlist POST	<item_id>, userId:<user_id>	201 Created with wishlist item added	201 Created, success:true, itemId:<item_id> added to wishlist	Pass
2	/api/wishlist POST	<item_id>, userId:<user_id>	200 OK, no duplicate items added	200 OK, success:true, duplicate prevented	Pass
3	/api/wishlist GET	userId:<user_id>	200 OK with list of wishlist items	200 OK, success:true, items:[jacket, Denim Shirt]	Pass
4	/api/wishlist/:id DELETE	wishlistItemId:<wishlist_id>	200 OK, item removed from wishlist	200 OK, success:true, item removed	Pass

10. Admin Module

The following tables summarize the documented test cases for the admin module based on the latest project test case set.

Unit Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
27	adminOnly middleware	req.user is undefined	401 Not authenticated	401 Error, message: Not authenticated	Pass
28	adminOnly middleware	req.user.role = user	403 Admin access only	403 Error, message: Admin access only	Pass
29	adminOnly middleware	req.user.role = admin	middleware should call next()	next() called successfully	Pass
30	updateItemStatus controller	body: { block: invalid }	400 Error, block must be true or false	400 Error, message: block must be true or false	Pass
31	updateItemStatus controller	valid item id, body: { block: true }	item status updated to blocked	success: true, data: { isBlocked: true }	Pass

Integration Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
1	/api/admin/dashboard GET	valid admin token	200 OK with dashboard statistics	200 OK, success: true, data: { totalUsers, totalItems, totalSwaps, totalReviews }	Pass
2	/api/admin/dashboard GET	missing Authorization token	401 Unauthorized	401 Error, message: Invalid/expired token	Pass
3	/api/admin/users GET	valid admin token	200 OK with paginated list of normal users	200 OK, success: true, page: 1, totalUsers: X, data: [users...]	Pass
4	/api/admin/users?email=testuser@gmail.com GET	valid admin token, email of an existing normal user	200 OK with matching user details	200 OK, success: true, data: { name, email, role, accountStatus }	Pass
5	/api/admin/users?email=nouser@gmail.com GET	valid admin token, non-existing email	404 Error, User not found	404 Error, message: User not found	Pass
6	/api/admin/users POST	name: Test User; email: testuser@gmail.com; password: *****; role: user	201 Created with new user record	201 Created, success: true, message: User created successfully by admin, data: { name, email, role }	Pass
7	/api/admin/users POST	duplicate email of an existing user	400 Error, User already exists	400 Error, message: User already exists	Pass
8	/api/admin/users/:email PATCH	valid admin token, email: testuser@gmail.com	200 OK, user account suspended and all owned items blocked	200 OK, success: true, message: User suspended and all items blocked	Pass
9	/api/admin/users/:active/:email PATCH	valid admin token, email: testuser@gmail.com	200 OK, user account activated and all owned items unblocked	200 OK, success: true, message: User activated and all items unblocked	Pass
10	/api/admin/users GET	valid token of non-admin user	403 Forbidden, Admin access only	403 Error, message: Admin access only	Pass
11	/api/admin/items GET	valid admin token	200 OK with paginated list of all items	200 OK, success: true, totalItems: X, data: [items...]	Pass
12	/api/admin/items?title=shirt GET	valid admin token, title: shirt	200 OK with items filtered by title	200 OK, success: true, data: [filtered items...]	Pass
13	/api/admin/items?email=user@gmail.com GET	valid admin token, owner email: user@gmail.com	200 OK with items belonging to the specified user	200 OK, success: true, data: [user items...]	Pass
14	/api/admin/items/:id/block PATCH	valid admin token, itemId: <item_id>, body: { block: true }	200 OK, item status updated to blocked	200 OK, success: true, message: Item blocked, data: { isBlocked: true }	Pass
15	/api/admin/items/:id/block PATCH	valid admin token, itemId: <item_id>, body: { block: false }	200 OK, item status updated to unblocked	200 OK, success: true, message: Item unblocked, data: { isBlocked: false }	Pass
16	/api/admin/items/:id/block PATCH	valid admin token, itemId: <item_id>, body: { block: yes }	400 Error, block must be true or false	400 Error, message: block must be true or false	Pass
17	/api/admin/items/:id DELETE	valid admin token, itemId: <item_id>	200 OK, item removed successfully	200 OK, success: true, message: Item removed	Pass

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
				successfully	
18	/api/admin/items/:id DELETE	valid admin token, itemId: fakeId	404 Error, Item not found	404 Error, message: Item not found	Pass
19	/api/admin/swaps GET	valid admin token	200 OK with list of swaps	200 OK, success: true, totalSwaps: X, data: [swaps...]	Pass
20	/api/admin/swaps?status=PENDING GET	valid admin token, status: PENDING	200 OK with only pending swaps	200 OK, success: true, data: [pending swaps...]	Pass
21	/api/admin/swaps?requesterEmail=user@gmail.com GET	valid admin token, requesterEmail: user@gmail.com	200 OK with swaps filtered by requester	200 OK, success: true, data: [filtered swaps...]	Pass
22	/api/admin/reviews GET	valid admin token	200 OK with list of all reviews	200 OK, success: true, totalReviews: X, data: [reviews...]	Pass
23	/api/admin/reviews?revieweeEmail=reviewee@gmail.com GET	valid admin token, revieweeEmail: reviewee@gmail.com	200 OK with reviews filtered by reviewee	200 OK, success: true, data: [filtered reviews...]	Pass
24	/api/admin/reviews/:id DELETE	valid admin token, reviewId: <review_id>	200 OK, review deleted successfully	200 OK, success: true, message: Review deleted successfully	Pass
25	/api/admin/reviews/:id DELETE	valid admin token, reviewId: fakeId	404 Error, Review not found	404 Error, message: Review not found	Pass
26	/api/admin/reviews GET	valid admin token	200 OK with linked item details populated in review data	200 OK, success: true, itemId: { _id, title, images }	Pass

Performance Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
32	/api/admin/dashboard GET	5 requests per second for 30 seconds using Artillery	API should respond successfully without crashing under load	Requests completed successfully with stable responses under test load	Pass
33	/api/admin/items GET	multiple concurrent requests using Artillery	API should return item list consistently under load	200 OK responses received consistently under concurrent load	Pass

11. System Module

The following tables summarize the documented test cases for the system module based on the latest project test case set.

Performance Test Cases

Test ID	Function	Inputs / Scenario	Expected Output	Real Output	Pass/Fail
1	/api/wishlist POST	100 concurrent users adding items	All requests processed within acceptable time (<2s)	Avg response time: 1.5s, no failures	Pass
2	/api/wishlist GET	500 requests per minute	Stable response, no server crash	Response time stable, no errors	Pass
3	/api/swaps/request POST	200 concurrent swap requests	All swaps created without data loss	All requests successful, no data inconsistency	Pass
4	/api/swaps/:id/track GET	1000 tracking requests	Fast response (<1s) with correct status	Avg response time: 0.8s	Pass
5	System Load Test	1000+ active users (wishlist + swaps combined)	System remains stable under load	Minor latency increase, system stable	Pass

12. Testing Instructions

How to run unit tests: Navigate to the backend or relevant package directory and run the configured unit test command, for example `npm test`. Ensure the necessary environment variables are loaded before execution.

How to run integration tests: Start the backend service and connect it to the test database or configured MongoDB instance. Execute the integration test suite using the project testing command and verify both success and error scenarios.

How to run performance tests: Use Artillery or a similar load-testing tool with a prepared YAML or JSON scenario file. Run the tests against the backend base URL and record latency, throughput, and error rates for inclusion in the report.

Testing environment configuration: Node.js 18+, NPM, MongoDB local instance or Atlas connection, backend environment variables, and any test authentication tokens or seeded records required for endpoint validation.